

Programación en Python

Clase 12: Git & GitHub

03/06/2026

Git, GitHub... ¿qué son?

Git, GitHub... ¿qué son?

Git es un sistema de control de versiones distribuido.

Git, GitHub... ¿qué son?

Git es un sistema de control de versiones distribuido.

GitHub es una plataforma en línea para alojar proyectos gestionados con Git.

Git, GitHub... ¿qué son?

Git es un sistema de control de versiones distribuido.

GitHub es una plataforma en línea para alojar proyectos gestionados con Git.

Git puede usarse sin GitHub.

Git, GitHub... ¿qué son?

Git es un sistema de control de versiones distribuido.

GitHub es una plataforma en línea para alojar proyectos gestionados con Git.

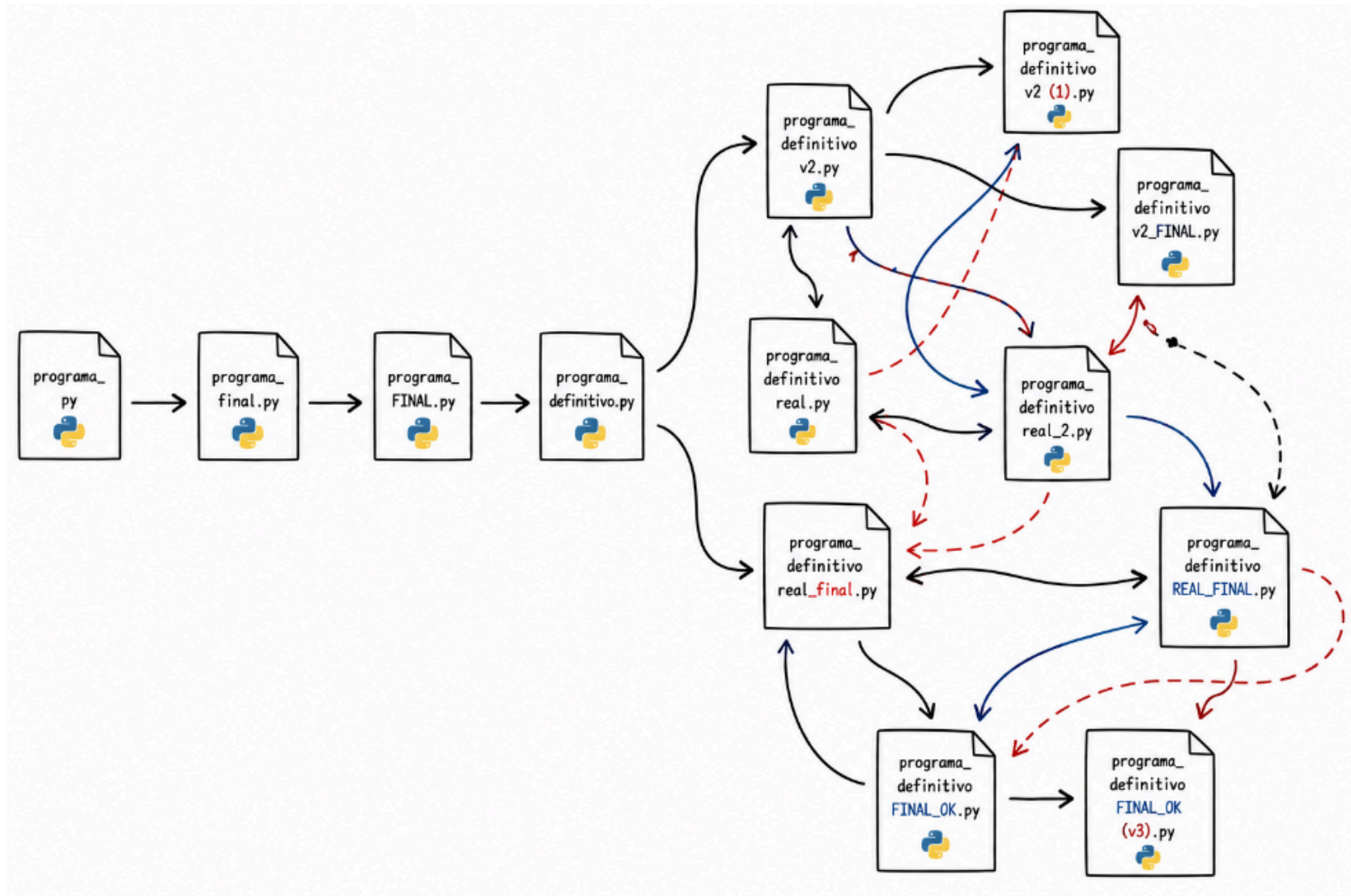
Git puede usarse sin GitHub.

GitHub no reemplaza a Git, lo complementa.

Git

A quién no le pasó

A quién no le pasó



Sistema de Control de Versiones (VCS)

Un **VCS** permite:

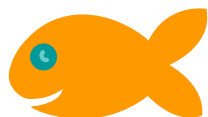
- Mantener un registro del historial de cambios.
- Documentar **qué** cambió y **cuándo** cambió.
- Volver a versiones anteriores si algo sale mal.
- Coordinar el trabajo cuando participan varias personas.

Sistema de Control de Versiones (VCS)

Un **VCS** permite:

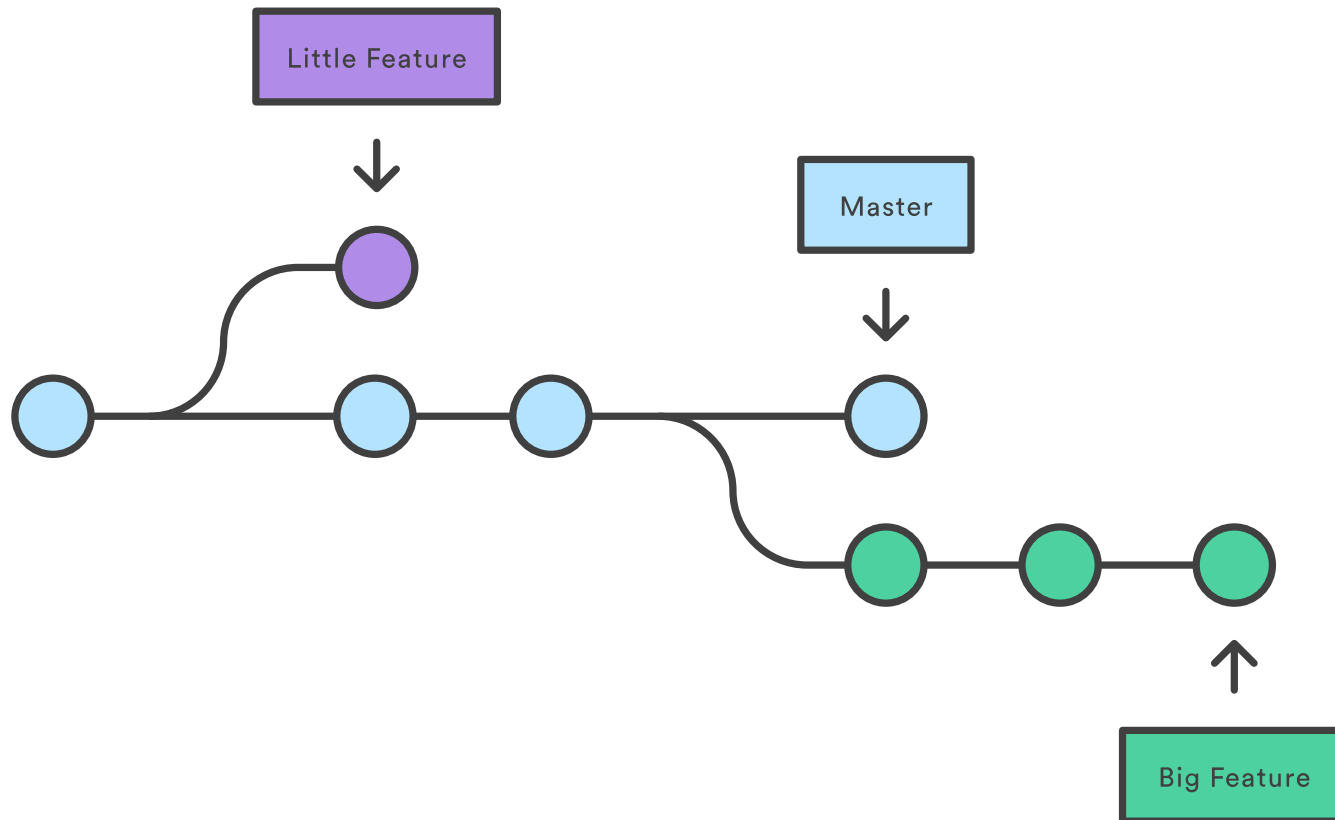
- Mantener un registro del historial de cambios.
- Documentar **qué** cambió y **cuándo** cambió.
- Volver a versiones anteriores si algo sale mal.
- Coordinar el trabajo cuando participan varias personas.

Algunos ejemplos:



El proyecto como historial

Git guarda “fotografías” del estado del proyecto en distintos momentos.



Cada punto representa un **commit**: una instantánea de los cambios guardados.

Sobre Git

Existen distintos sistemas de control de versiones. Lo distintivo de Git es que es:

- **Distribuido:** cada persona tiene una copia completa del repositorio y de su historial.
- **Eficiente:** las operaciones suelen ser rápidas y el almacenamiento es liviano.
- **Completo:** permite registrar cambios, comparar versiones, crear ramas y fusionarlas.
- **Open source:** es libre y sigue evolucionando desde su aparición en 2005.

Página oficial: <https://git-scm.com>

Instalación

Demostración en vivo

Git Bash

Git Bash es una consola para usar Git y comandos de terminal estilo Linux dentro de Windows.

Comandos básicos de Bash

```
1  ls
2  cd <directorio>
3  cd ..
4  pwd
5  mkdir <nombre>
6  touch <nombre>
7  rm <nombre>
8  cp <nombre> <directorio>
9  mv <nombre> <directorio>
```

Configuración mínima

Todas las acciones en Git necesitan un autor.

Se utiliza el comando `git config` de la siguiente manera:

```
git config --global user.name <nombre>  
git config --global user.email <email>
```

Si se omite el *flag* `--global`, la configuración aplica solo al repositorio donde se realiza.

Para ver qué otras cosas se pueden hacer con `git config`:

```
git config -h
```

Repositorio

Un **repositorio** es un lugar donde se guarda un proyecto, incluyendo sus archivos, carpetas y el historial de cambios.

Repositorio

Un **repositorio** es un lugar donde se guarda un proyecto, incluyendo sus archivos, carpetas y el historial de cambios.

Git se encarga del seguimiento del historial de cambios y nos permite navegar entre versiones de nuestro proyecto.

Repositorio

Un **repositorio** es un lugar donde se guarda un proyecto, incluyendo sus archivos, carpetas y el historial de cambios.

Git se encarga del seguimiento del historial de cambios y nos permite navegar entre versiones de nuestro proyecto.

Las **ramas** hacen posible trabajar de forma colaborativa e independiente: se pueden realizar cambios, probar ideas o corregir errores sin afectar directamente el proyecto principal.

Repositorio

Un **repositorio** es un lugar donde se guarda un proyecto, incluyendo sus archivos, carpetas y el historial de cambios.

Git se encarga del seguimiento del historial de cambios y nos permite navegar entre versiones de nuestro proyecto.

Las **ramas** hacen posible trabajar de forma colaborativa e independiente: se pueden realizar cambios, probar ideas o corregir errores sin afectar directamente el proyecto principal.

Git se encarga de que la **fusión** de cambios sea amena y segura.

Inicialización de un repositorio

Para inicializar un repositorio se debe ejecutar `git init` en la raíz del proyecto:

```
git init
```

```
Initialized empty Git repository in /prueba/.git/
```

Inicialización de un repositorio

Para inicializar un repositorio se debe ejecutar `git init` en la raíz del proyecto:

```
git init
```

```
Initialized empty Git repository in /prueba/.git/
```

Se crea una carpeta oculta llamada `.git` que contiene todas las referencias asociadas al sistema de control de versiones.

```
ls .git
```

```
branches  config  description  HEAD  hooks  info  objects  refs
```

Guardado de cambios

En Git no alcanza con modificar archivos:

1. Hay que elegir qué cambios guardar.
2. Luego, hay que “guardarlos” explícitamente

Guardado de cambios

En Git no alcanza con modificar archivos:

1. Hay que elegir qué cambios guardar.
2. Luego, hay que “guardarlos” explícitamente

```
git status  
git add <archivo>  
git add .  
git commit -m "<mensaje>"
```

- `git status` muestra el estado actual del repositorio.
- `git add` agrega cambios al área de *stage*.
- `git commit` crea la fotografía del estado preparado.
- El mensaje debe describir de manera clara qué se guardó.

El área de *stage*

El área de *stage* es una zona intermedia donde se preparan los cambios del próximo commit.

```
archivos modificados -> stage -> commit
```

Git solo guarda en el commit los cambios que fueron agregados con `git add`.

Consulta de estado

Para saber qué está pasando en el repositorio usamos principalmente:

```
git status  
git log
```

- `git status` muestra archivos modificados, eliminados, agregados o no rastreados.
- También indica qué cambios ya están en *stage* y cuáles no.
- `git log` muestra la lista de commits realizados.
- Cada commit tiene un **hash** que lo identifica de manera única.

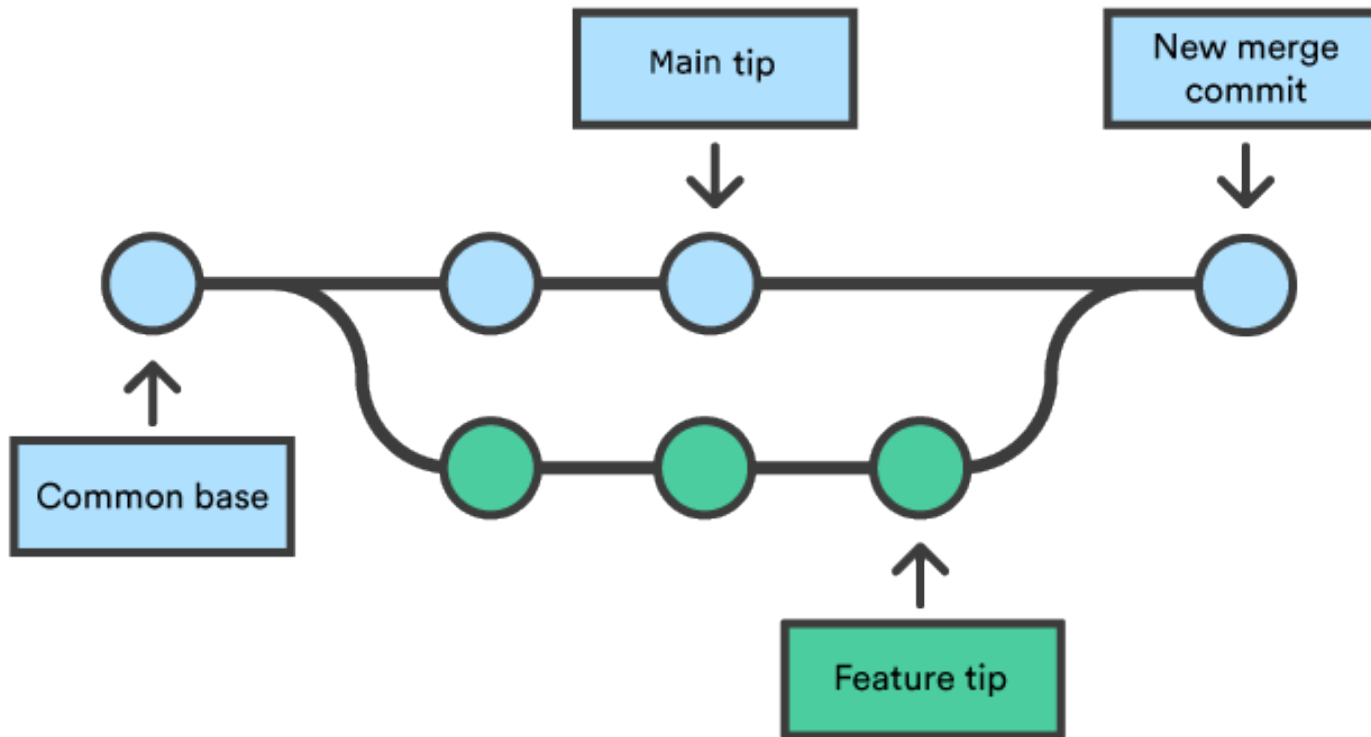
Ramas

Una **rama** es una línea de desarrollo independiente dentro del repositorio.

- Permite trabajar en una parte del proyecto sin afectar directamente a otras.
- Sirve para probar ideas, desarrollar funcionalidades o corregir errores.
- Cada rama tiene su propio conjunto de cambios y commits.
- Cuando el trabajo está listo, puede fusionarse con otra rama.

Ramas

Una rama parte de un *commit*, avanza de manera separada y luego puede integrarse.



Manipulación de ramas

Comandos básicos para crear y movernos entre ramas:

```
git branch  
git branch <nombre_rama>  
git switch <nombre_rama>  
git switch -c <nombre_rama>  
git checkout -b <nombre_rama>
```

- `git branch` lista las ramas existentes.
- `git branch <nombre_rama>` crea una rama nueva.
- `git switch <nombre_rama>` cambia a otra rama.
- `git switch -c` crea una rama y se mueve a ella.

Fusión de ramas

En algún momento necesitamos integrar el trabajo de una rama en otra.

Usaremos `git merge`.

```
git merge <nombre_rama>
```

- Hay que estar situados en la rama de destino.
- Git toma los cambios de la otra rama y los aplica sobre la actual.
- La fusión combina los historiales de ambas líneas de trabajo.
- Después se puede verificar el resultado con `git status` o `git log`.

Eliminación de ramas

Cuando una rama ya cumplió su objetivo, puede eliminarse:

```
git branch -d <nombre_rama>  
git branch -D <nombre_rama>
```

- -d intenta borrar la rama de manera segura.

Eliminación de ramas

Cuando una rama ya cumplió su objetivo, puede eliminarse:

```
git branch -d <nombre_rama>  
git branch -D <nombre_rama>
```

- -d intenta borrar la rama de manera segura.
- -D fuerza la eliminación.

Eliminación de ramas

Cuando una rama ya cumplió su objetivo, puede eliminarse:

```
git branch -d <nombre_rama>  
git branch -D <nombre_rama>
```

- -d intenta borrar la rama de manera segura.
- -D fuerza la eliminación.
- Conviene borrar ramas cuando ya fueron integradas o dejaron de ser útiles.

El archivo `.gitignore`

A veces hay archivos que no queremos incluir en los commits:

- Información delicada que no debería versionarse.
- Archivos temporales.
- Resultados generados automáticamente.
- Configuraciones locales.

El archivo `.gitignore`

A veces hay archivos que no queremos incluir en los commits:

- Información delicada que no debería versionarse.
- Archivos temporales.
- Resultados generados automáticamente.
- Configuraciones locales.

El archivo `.gitignore` se ubica en la raíz del proyecto y contiene **reglas de exclusión**. Por ejemplo:

```
archivo_temporal.txt
carpeta_temporal/
*.log
**/temp
```

El archivo `.gitignore`

A veces hay archivos que no queremos incluir en los commits:

- Información delicada que no debería versionarse.
- Archivos temporales.
- Resultados generados automáticamente.
- Configuraciones locales.

El archivo `.gitignore` se ubica en la raíz del proyecto y contiene **reglas de exclusión**. Por ejemplo:

```
archivo_temporal.txt
carpeta_temporal/
*.log
**/temp
```

El archivo `.gitignore` sí debe agregarse al repositorio.